ZG519 NLP APPLICATIONS

Course Overview & NLP Applications

Ten application areas, easiest to hardest, and why the easy ones still run in production

BUILT FROM: THE SESSION 1 SLIDES, COURSE HANDOUT AND RECORDING · EVERY EXAMPLE WORKED OUT IN FULL

How to use this companion. This companion explains Session 1 from scratch, in plain English. Every idea starts with an everyday picture. Then the mechanism is built up one step at a time. Every example is worked out in full. Small labels mark the boxes. **THE INTUITION** states the core idea in one breath. **EVERYDAY PICTURE** ties it to real life before any mechanism. **WORKED EXAMPLE**, on a grey background, walks a real task through with no skipped steps. **WATCH OUT** flags the common mistake. **KEY TAKEAWAY** is the one sentence to remember. Read it in order; each section leans on the one before.

1 Try it first: what does one messy sentence need?

Before any course map, feel how many separate NLP steps a single real message already demands.

Here is a WhatsApp message. It is exactly the kind an NLP system meets daily:

> *"Bro, idhu kashtam (this is hard) but deadline tmrw, pls extend!!"*

Before reading on, picture handing this sentence straight to a translation tool, with no other processing. Ask three questions. Does the tool know *which words are English and which are Tamil*? Does it know the writer is *stressed, asking for a favour*, not stating a neutral fact? Does it even know "tmrw" means "tomorrow"? A raw translation tool assumes clean, single-language, correctly-spelled input. This sentence is none of those three things, and real text almost never is.

That gap names three separate jobs, each its own application area this semester. First, **spelling and grammar correction** turns "tmrw" into "tomorrow" before anything downstream can use it cleanly. Second, **language identification** has to spot that "idhu kashtam" is Tamil, mixed into an English sentence — called **code-switching**. Third, **sentiment analysis** has to read the stressed, pleading tone from "pls extend!!", not just the dictionary meaning of the words. One short message, three applications, before any translation or question-answering

even starts. That ordering — clean first, understand next, act last — is the backbone of this entire course.

2 Sixteen sessions, one map

Every session this semester, and exactly how the two exams are weighted, straight from the official handout.

#	Topic	Sub-topics
1	Introduction	Overview of interesting NLP applications
2	Spellcheck & grammar check	Techniques for automatic checkers
3	Retrieval-Augmented Generation	Naïve RAG, Graph RAG, Agentic RAG
4	Question answering	Agentic AI + Gen AI techniques in QA
5	Conversational AI	Hybrid systems: LLM + RAG + Agentic AI
6	Sentiment analysis	Methods; neural networks for sentiment
7	Privacy & ethics in NLP	Data, model, and deployment issues
8	Pre-midsem review	Review of Sessions 1–7
9	Machine translation	Statistical approaches
10	Machine translation	Neural approaches
11	Machine translation	Indic language translation
12	Information extraction	Named entity recognition
13	Information extraction	Relation extraction
14	Information extraction	Events and time extraction
15	Scalable & efficient NLP	Compression, inference, deployment
16	Post-midsem review	Review of Sessions 9–15

All sixteen contact sessions, compressed.

Component	Weight	Notes
2 quizzes (pre- & post-midsem)	10%	Open on mobile; mark dates the moment they post
Programming assignment(s)	25%	Groups of 4, end-to-end applications
Situated learning	5%	Apply one application to your own workplace
Mid-semester exam	30%	Closed book, Sessions 1–8 only
Comprehensive final exam	35%	Open book, all topics

How the course is graded.

WATCH OUT

The exam that rewards memorisation is the mid-sem, not the final. The mid-sem is closed book and covers only Sessions 1–8: spelling, RAG basics, QA, conversational AI, sentiment, and ethics. Learn those as you go. The comprehensive final is open book, so it rewards fluency with the tools, not recall.

3 Ten applications, one difficulty ladder

The professor orders every application by difficulty on purpose. Knowing when a cheap classical method beats an expensive agent is itself an exam skill.

A fair question up front. Large language models can do almost anything with text, so why does this course still teach ten separate, old-fashioned-sounding applications? Because production systems are **hybrid**. The simple method is often cheaper and faster. It is also often just as accurate as the fancy one. Knowing which to reach for is the actual skill this course builds.

THE INTUITION

Every later era of a technology keeps using the earlier one wherever it is good enough. A spam filter does not need an agent. A multi-hop research question might. The ladder below is a map of *when* to reach for more machinery, not a claim that newer always wins.

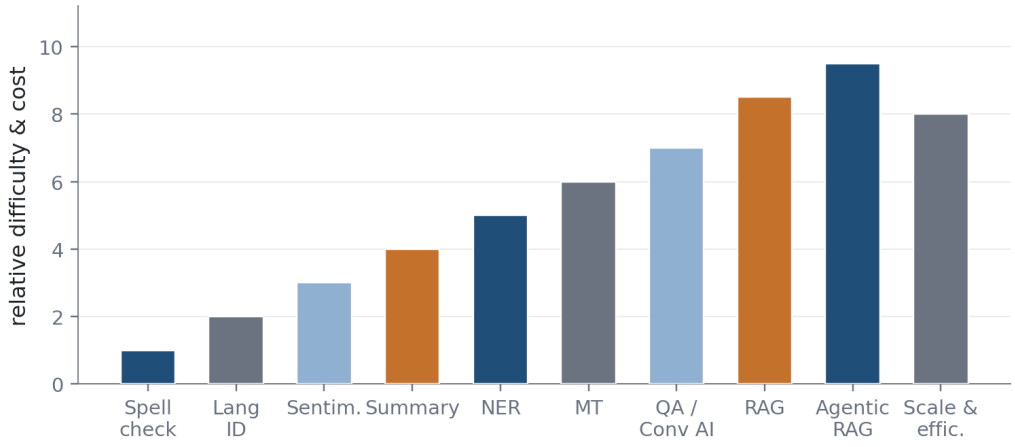


Figure 1. Ten applications, ranked by how much machinery (and cost) they typically need, from a near-perfect classical method to a full agent loop.

1 · Spell & grammar check — the baseline. Classic statistical methods (Naive-Bayes-style) already get this close to 100% right, no large language model required. It still matters: it is the first filter in front of every later application. A clean audio recording has to exist, the same way, before anyone can transcribe it.

2 · Language identification & code-switching. You must know the language before you can translate, tag, or search it. The genuinely hard case is **code-switching**: one sentence

mixing two languages, exactly like the WhatsApp message in Section 1.

WORKED EXAMPLE · *tagging the code-switched message word by word*

Step 1 Read each word and ask: which language's vocabulary does this word belong to?

Step 2 "Bro,", "but", "deadline", "tmrw" — English spellings and grammar, even where "tmrw" is a shortened spelling.

Step 3 "idhu", "kashtam" — Tamil words ("this", "hard"), written in English letters, a common style called transliteration.

Step 4 Tag each token EN or TA. The output is not one label for the whole sentence. It is a label *per word*. That is exactly why code-switching breaks single-language detectors: they expect one answer for the whole input.

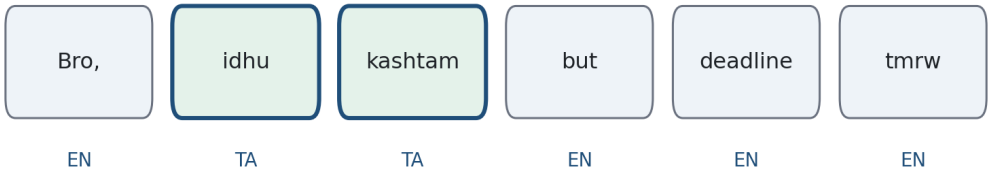


Figure 2. Language identification on a code-switched sentence: each word gets its own EN/TA tag, not one tag for the whole sentence.

3 · Sentiment analysis. Classifying text as positive, negative, or neutral — a movie review, a product review, a political statement. The signal comes from word choice, punctuation ("!!"), and emphasis, not just the dictionary meaning of each word.

4 · Summarization: extractive versus abstractive. **Extractive** summarization copies real sentences straight from the source. **Abstractive** summarization writes new sentences that capture the gist, in the system's own words. It is much harder, because it requires actually following the content, not just picking good excerpts.

5 · Information extraction — entities, relations, events. Covered in full depth in Section 4 below: turning a plain sentence into structured facts a database can store.

6 · Machine translation. A headline topic this run, with three dedicated sessions. Statistical MT comes first, then neural MT, then Indic-language MT specifically, tied to national efforts like AI4Bharat and Sarvam AI.

7 · Question answering & conversational AI. A huge open-source and commercial tooling ecosystem lives here. The course builds these as hybrid systems — LLM plus RAG plus Agentic AI. A bare LLM alone hallucinates on specific factual questions.

8 · RAG — naïve, graph, and agentic. The hardest application area in the course. Covered in full depth in Section 5 below.

9 · Agentic AI & guardrails. Agentic systems carry a real cost. Production teams do not reach for them by default; they escalate only where simpler methods fail. Whenever agents are used, guardrails and a human in the loop are treated as non-negotiable, not optional extras.

10 · Scalable & efficient NLP systems. New to this run. It was added because a method that works beautifully on a small test set can fail in production. Latency, scale, and cost problems often never show up in a notebook. Session 15 covers model compression and inference optimisation directly.

> **KEY TAKEAWAY**
>
> *Ten applications, one ladder: near-perfect classical methods at the easy end, a full agent loop with guardrails at the hard end. The skill this course teaches is not "always use the newest method." It is knowing which rung of the ladder a given problem actually needs.*

4 Worked example: turning a sentence into structured facts

Information extraction has three layers: who is mentioned, what did they do to whom, and when.

Named entity recognition (NER) finds proper nouns — people, places, organisations — in running text. **Relation extraction** then connects two entities it found. **Event extraction** adds the "when". All three lean on knowing the **part of speech** of each word, since named entities typically begin with a proper noun.

> **EVERYDAY PICTURE**
>
> This is exactly what IMDb does automatically to a movie review. It tags the cast, the director, and the studio it mentions — entities. It works out who directed whom — a relation. It notes the release date the review refers to — an event. All of this comes from one paragraph of free text.

> **WORKED EXAMPLE** · *extracting structured facts from one sentence*
>
> Sentence: "Google acquired DeepMind in 2014 for Rs. 4,000 crore."
>
> **Step 1** **Tokenise & tag part of speech.** Split into words; mark "Google" and "DeepMind" as proper nouns, "acquired" as a verb, "2014" and "4,000" as numbers.
>
> **Step 2** **Named entity recognition.** Label each proper noun and number span with its entity type: `Google = ORG`, `DeepMind = ORG`, `2014 = DATE`, `Rs. 4,000 crore = MONEY`.
>
> **Step 3** **Relation extraction.** The verb "acquired" links the two ORG entities in a fixed

direction: `acquired(Google, DeepMind)`. Google is the buyer. DeepMind is the one bought, not the reverse.

Step 4 **Event extraction.** Attach the DATE and MONEY entities to that relation as the event's details. The result: an acquisition event, subject Google, object DeepMind, time 2014, cost Rs. 4,000 crore.

The output is no longer a sentence. It is a row a database can store: one event, four typed fields, ready to query.

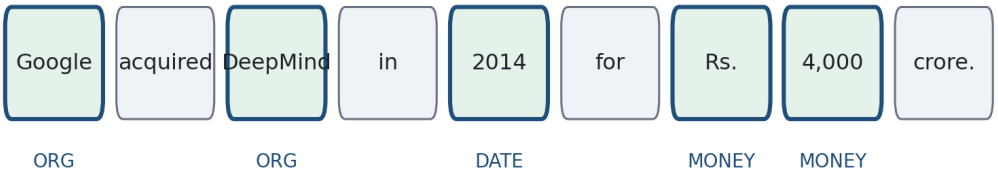


Figure 3. Named entity tags on the acquisition sentence: ORG, DATE, and MONEY spans, each found independently before being linked into one event.

WATCH OUT

NER alone does not tell you *who acquired whom*. "Google acquired DeepMind" and "DeepMind acquired Google" contain the exact same two entities. Only relation extraction, which reads the verb and its direction, tells the two apart. Entities, relations, and events are three separate jobs, not one.

KEY TAKEAWAY

Information extraction builds structured facts in layers. Find the entities first. Link pairs of them into relations. Then attach time and detail to make a full event. Each layer depends on the one before it being right.

5 Worked example: the RAG ladder — naïve, graph, agentic

Three ways to answer one question, each fixing the previous one's blind spot on multi-step questions.

Retrieval-Augmented Generation (RAG) answers a question in two steps. First it retrieves relevant text. Then it generates an answer grounded in that text, instead of relying only on what the model memorised during training.

EVERYDAY PICTURE

Naïve RAG is a friend who looks up one relevant fact before answering your question. Graph RAG is a friend who also knows how facts connect to each other. They can answer "how is X related to Y" by walking a web of relationships, not just reading isolated snippets. Agentic RAG is that same friend deciding, mid-conversation, "actually, I should look that up again before I answer this follow-up." Retrieval becomes a decision for them to make, not a fixed, one-time step.

WORKED EXAMPLE · *answering a two-hop question three different ways*

Question: "What company did DeepMind's co-founder previously work at before DeepMind was acquired by Google?"

Step 1 **Naïve RAG.** Retrieve the single text chunk most similar to the question — likely a chunk about the Google–DeepMind acquisition. That chunk may never mention the co-founder's earlier job. The system answers confidently from an incomplete chunk, or admits it does not know.

Step 2 **Graph RAG.** The underlying knowledge graph stores typed edges: `founded(Person, DeepMind)`, `acquired(Google, DeepMind)`, `previously_worked_at(Person, Company)`. Following the `Person` node across all three edges answers the question in one lookup. The connection between facts is explicit in the graph, not buried in prose.

Step 3 **Agentic RAG.** With plain text chunks and no graph, the system reasons for itself. "The first chunk names the co-founder, but not their earlier job. I need a second, different retrieval." It retrieves again, this time searching for that person's name plus "previously worked at". Two retrieval rounds, chosen by the system itself, replace the one graph lookup.

All three reach the same correct answer here. Naïve RAG is the one that quietly fails on real two-hop questions like this one. It never gets a second chance to look further.

WAIT — REALLY?

Agentic RAG does not use a smarter retrieval model than naïve RAG. The retrieval step is identical. What changed is that retrieval now sits *inside a loop* the system controls. That is the same reason-act-observe pattern that turns a chatbot into an agent elsewhere in this course.



Figure 4. Agentic RAG as a loop: the system decides, after each retrieval, whether it has enough evidence yet or needs to look again.

> **KEY TAKEAWAY**
>
> *Naïve RAG retrieves once, fixed. Graph RAG retrieves across explicit connections between facts. Agentic RAG retrieves as many times as the system itself decides it needs to. Multi-hop questions are where naïve RAG's one-shot limit shows up first.*

Where it's used. This same ladder explains a common failure. A company's internal search bot often answers simple FAQs correctly. It can still fail on "compare last year's and this year's policy on X." That is a two-hop question, and plain, single-pass RAG was never built to answer it.

Self-test

1. Why does spelling and grammar correction run before almost every other NLP application, not after?

Every later step — language ID, sentiment, NER, translation — assumes reasonably clean input text. An uncorrected "tmrw" or a mis-split sentence makes every downstream step less accurate.

2. Why do single-language detectors fail on code-switched text?

They are built to output one label for the whole input. A code-switched sentence genuinely needs a different label per word, so a single whole-sentence answer is wrong by construction.

3. In "Google acquired DeepMind", what does relation extraction add that named entity recognition alone cannot tell you?

Direction. NER finds that Google and DeepMind are both organisations, but only the verb-driven relation extraction step tells you *which one acquired which*.

4. Why does naïve RAG struggle on a two-hop question like the DeepMind co-founder example?

It retrieves only once. If the single best-matching chunk does not contain every fact the answer needs, there is no second chance to look further. Graph RAG has explicit connections instead, and agentic RAG can simply retrieve again.

5. Why does the mid-semester exam reward memorisation more than the comprehensive final does?

The mid-sem is closed book and covers only Sessions 1–8. The comprehensive final is open book and covers everything, so it rewards fluency using the material, not recalling it from memory.

Pocket card: Session 1 in one page

KEY TAKEAWAY

The whole session, compressed.

- *One messy real sentence already needs three separate applications — **spelling correction**, **language ID** (code-switching), and **sentiment**. Clean first, understand next, act last.*
- *Sixteen sessions, two exams: mid-sem is **closed book**, Sessions 1–8 only; the final is **open book**, all topics.*
- *Ten applications form one **difficulty ladder**, from near-perfect classical spell-check to a full agent loop with guardrails. Production systems are **hybrid**: use the cheapest method that works.*
- ***Information extraction** builds structured facts in layers: entities → relations → events. Each layer needs the one before it to be correct.*
- *The **RAG ladder**: naïve RAG retrieves once. Graph RAG follows explicit connections between facts. Agentic RAG decides, on its own, whether to retrieve again. Multi-hop questions expose naïve RAG's one-shot limit first.*

Glossary & symbols

Key terms.

code-switching one sentence or message that mixes two or more languages, often within a single line.

extractive summary a summary built only from sentences copied verbatim from the source text.

abstractive summary a summary written in new words that capture the gist, without copying sentences.

named entity recognition (NER) finding spans of text that name a person, place, organisation, date, or amount.

relation extraction linking two entities already found by NER into a typed, directional connection (who did what to whom).

event extraction attaching time and other detail to a relation to describe a full real-world event.

RAG Retrieval-Augmented Generation: retrieve relevant text, then generate an answer grounded in it.

graph RAG RAG where retrieval walks explicit, typed connections between facts in a knowledge graph, not flat text chunks.

agentic RAG RAG where the system itself decides when and what to retrieve, possibly

retrieving more than once.

hybrid system a production system that deliberately mixes cheap classical methods with expensive LLM/agentic methods, by task.

guardrail a hard limit placed on an agent's actions, to keep autonomy inside a safe, checked boundary.